游戏《杀戮尖塔》中卡牌“华丽收场”的形式化建模

哈基米

日期：2026 年 7 月 2 日

1 背景

形式化方法作为保障复杂系统正确性与可靠性的严谨数学手段，在硬件验证、分布式协议及安全关键软件等领域得到了广泛应用。随着现代电子游戏逻辑的复杂化，其内部状态机的流转、非确定性事件的处理以及策略搜索空间的膨胀，也逐渐成为形式化验证在大众软件领域的新兴应用场景之一。

由 Mega Crit 工作室制作的 Roguelike 卡牌游戏《杀戮尖塔》（Slay the Spire）具有严密的规则体系和极高的策略深度，在全球范围内售出了数百万份，今年《杀戮尖塔 2》一经发售即达到了超过 57 万的同时在线人数，是最经典的卡组构建游戏之一。

在游戏中，“静默猎手”（The Silent）作为以高效抽牌、弃牌为核心机制的角色，是设计最为优秀的角色之一；而其中，静默猎手的卡牌“华丽收场”（Grand Finale）是公认的整个游戏中设计最出色的卡牌，具有严苛的触发条件与强大的效果。该卡牌要求玩家在使用时，其抽牌堆的卡牌数量必须为 0，而如何通过当前手牌与抽牌堆的特定组合，利用“过牌”与“弃牌”的联动效应，在有限的能量与手牌上限的约束下，最终过空抽牌堆，达成“华丽收场”的触发状态，是一个典型的状态可达性验证问题。



图 1: 游戏中的华丽收场卡牌

2 模型假设

2.1 游戏规则

在《杀戮尖塔》游戏中，打出每张牌都需要玩家当前的能量大于卡牌所需能量，消耗对应能量并结算卡牌效果后，将该牌置入弃牌堆。玩家初始时具有 5 张手牌和 3 点能量，手牌上限为 10，目的即是在限制条件下，通过各种具有抽牌和弃牌的效果的卡牌，抽空抽牌堆中的所有牌，从而达到卡牌“华丽收场”的打出条件。

当需要抽牌的数量大于抽牌堆剩余卡牌时（例如抽牌堆中还剩 1 张牌时打出了一张需要抽 4 张牌的杂技 +），弃牌堆中的所有卡牌都将被随机洗入抽牌堆，这是一个高度随机的过程，具有 $O(N!)$ 的复杂度，会使模型的复杂度大幅上升。考虑到求解“华丽收场”残局的核心目标是精确抽空当前牌库，一旦触发洗牌，则意味着玩家失去了对牌库数量的精确控制，因此，我们在目前的简化模型中暂不考虑触发洗牌的情况，将所有会触发洗牌的情况视为无效。

同时，我们忽略玩家生命值、怪物意图、攻击伤害等不影响过牌运转的环境变量，只考虑卡牌集合、能量与抽牌堆的状态转移。

2.2 卡牌列表

我们考虑的卡牌如下：

- **白板 (Blank)** 耗能 1，没有效果，用于模拟运转过程中的无效手牌。
- **杂技 (Acrobatics)** 耗能 1，抽 3 张牌，随后从手牌中选择并弃置 1 张牌。
- **杂技 + (Acrobatics+)** 杂技的升级版，耗能 1，抽 4 张牌，随后从手牌中选择并弃置 1 张牌。
- **战术大师 + (Tactician+)** 无法打出。当被主动弃掉时，获得 2 点能量。
- **本能反应 (Reflex)** 无法打出。当被主动弃掉时，抽 2 张牌。
- **本能反应 + (Reflex+)** 无法打出。当被主动弃掉时，抽 3 张牌。
- **早有准备 + (Prepared+)** 耗能 0，抽 2 张牌，随后从手牌中选择并弃置 2 张牌。
- **独门技术 + (Expertise+)** 耗能 1，抽牌直至手牌数量达到 7 张。
- **计算下注 + (Calculated Gamble+)** 耗能 0，丢弃手中除该牌以外的所有牌，并抽取与弃牌数量相等的牌。
- **全神贯注 + (Concentrate+)** 从手中选择并弃置 2 张牌，获得 2 点能量。
- **华丽收场 (Grand Finale)** 0 耗能，当抽牌堆刚好没有任何牌时方可打出，对所有敌人造成 50 点伤害。在我们的 TLA+ 模型中，成功打出此牌即视为模型找到了解。



图 2: 游戏中的卡牌示例

3 模型设计

本模型的核心设计思路是将复杂的非确定性卡牌运转过程，映射为严格的有限状态机可达性问题，并通过 TLA+ 的不变量检测，通过“证伪”得到出牌策略。需要注意的是，这里得到的策略并不一定是最优策略（例如耗能最少的策略），但一定是路径最短的保证能打出华丽收场的策略之一，具体会在下文说明。

模型使用 TLA+ 语言。首先通过 `EXTENDS Integers, Sequences, FiniteSets` 引入了 TLA+ 的基础算术、序列运算与有限集运算库，以准备构建牌库序列和枚举弃牌组合。

在常量部分定义了模型所包含的核心卡牌集合。由于 TLA+ 不支持直接的字符串枚举作为类型，这里采用字符串常量集合来唯一标识每一张卡牌，每张卡牌的具体信息如上文所述。`MaxHandSize` 设置为 10。

接下来，模型通过 `VARIABLES` 关键字声明了核心状态变量，`hand`，和 `discard_pile` 分别为手牌和弃牌堆，因为手牌和弃牌堆在模型中没有顺序要求，所以采用多重集合的结构表示，大大压缩了 TLC 需要遍历的状态空间。`draw_pile` 为抽牌堆，与手牌不同的是，抽牌堆的顺序会影响未来的抽牌结果。因此，模型对抽牌堆采用序列数据结构，保证抽牌的先后次序。此外，`energy` 用于记录能量，`reshuffled` 用于记录是否发生洗牌，`grand_finale_played` 用于记录是否已经打出华丽收场。

模型还定义了一系列辅助函数，包括用于计算当前手牌数的 `HandSize`，用于操作多重集合的 `AddBags` 和 `SubBags`，`SeqToBag` 等。

实际游戏中，弃牌时选择哪张牌是最重要的战术决策之一，如果采用传统方法尝试让 AI 判断弃哪张牌，实现难度会很高，但在形式化方法中，我们可以使用 `ValidDiscards(b, n)` 直接遍历所有可能的弃牌分支。

最后，我们通过 `DoDraw` 实现了抽牌逻辑。在游戏中，超出手牌上限的抽牌动作仅仅会抽到手牌上限（例如手上有 9 张牌时打出一张杂技 +，会抽 2 张牌后选择弃一张牌），因此需要计算 `target_draw`，并判断是否触发洗牌。

接下来写入初始状态。

```
Init ==
    /\ energy = 3
    /\ reshuffled = FALSE
    /\ grand_finale_played = FALSE
    /\ hand = [EmptyBag EXCEPT !["GrandFinale"] = 1, !["
       CalculatedGamblePlus"] = 1,
                               !["ReflexPlus"] = 1, !["AcrobaticsPlus"] =
                                  1, !["Blank"] = 1]
    /\ discard_pile = EmptyBag
    /\ draw_pile = <<"Acrobatics", "Blank", "Blank", "Blank", "Blank", "
       Blank">>
```

以最复杂的杂技为例，具体出牌动作的实现需要先判定触发条件：

```
    /\ ~reshuffled          -- 尚未发生过洗牌失败
    /\ hand[card_name] > 0 -- 手中有这张牌
    /\ energy >= 1          -- 至少有1点能量
```

接下来，支付成本，并执行抽牌动作，同时判断是否发生了洗牌：

```
    h1 == [hand EXCEPT ![card_name] = @ - 1]    -- 从手牌移除本卡
    e1 == energy - 1                             -- 消耗1点能量
    d_res1 == DoDraw(draw_pile, draw_amt, HandSize(h1))

    IF d_res1.is_reshuffled THEN
        /\ reshuffled' = TRUE
        /\ UNCHANGED <<hand, energy, discard_pile, draw_pile, grand_finale_
            played>>
```

抽牌成功后，抽到的牌是序列形式，我们将其转换为多重集合，并与移除了所打出牌杂技后的手牌合并，得到弃牌前的手牌 h2。随后，我们使用 TLA+ 的存在量词 \E 遍历所有可能的弃牌决策，并结算对应的弃牌收益。

```
    h2 == AddBags(h1, SeqToBag(d_res1.drawn))
    \E d_bag \in ValidDiscards(h2, 1):
    h3 == SubBags(h2, d_bag)                     -- 从手牌中移除弃牌
    e2 == e1 + EnergyGainFromBag(d_bag)          -- 弃掉战术大师+得能量
    ex_draw == DrawGainFromBag(d_bag)            -- 弃掉本能反应得抽牌数
    d_res2 == DoDraw(d_res1.rem, ex_draw, HandSize(h3))
```

同理，弃牌后也需要判断一次是否发生了抽牌。判断完毕后，进行状态更新。

```
    IF d_res2.is_reshuffled THEN
        /\ reshuffled' = TRUE
        /\ UNCHANGED <<...>>
    ELSE
        /\ hand' = h4                                -- 最终手牌
        /\ energy' = e2                              -- 最终能量
        /\ draw_pile' = d_res2.rem                   -- 剩余牌库
        /\ reshuffled' = FALSE
        /\ discard_pile' = AddBags(discard_pile,
             AddBags(d_bag, [EmptyBag EXCEPT ![card_name]=1]))
        /\ grand_finale_played' = grand_finale_played
```

模型会为每个状态判断是否满足 grand_finale_played，若满足，则会改变不变量 GameNotWon，程序会报告反例，并给出一条初始状态到达反例的操作序列，我们即可从中得到出牌顺序。

```
Stall ==
/\ UNCHANGED <<hand, draw_pile, discard_pile, energy, reshuffled, grand_
   finale_played>>

Next == PlayGrandFinale \/ PlayBlank \/ PlayAcrobatics \/
   PlayAcrobaticsPlus \/
    PlayExpertisePlus \/ PlayPreparedPlus \/ PlayConcentratePlus \/
       PlayCalculatedGamblePlus \/ Stall
```

```
GameNotWon == ~grand_finale_played
```

完整源代码见附件。

4 模型实战

我们可以通过一局实际游戏判断模型的有效性。在这场战斗中，初始手牌为早有准备 +，战术大师 +，独门技术 +，防御（即白板卡牌），全神贯注 +；抽牌堆依次为华丽收场 +，防御，防御，本能反应 +，杂技 +，防御。这里的华丽收场 + 和普通的华丽收场只有 10 点伤害区别，对实验没有影响。

将初始状态输入模型，使用 TLC 进行验证：

```
Init ==
    /\ energy = 3
    /\ reshuffled = FALSE
    /\ grand_finale_played = FALSE
    /\ hand = [EmptyBag EXCEPT ![“PreparedPlus”] = 1, ![“TacticianPlus”] =
       1, ![“ExpertisePlus”] = 1, ![“Blank”] = 1, ![“ConcentratePlus”] = 1]
    /\ discard_pile = EmptyBag
    /\ draw_pile = <<“GrandFinale”, “Blank”, “Blank”, “ReflexPlus”, “
       AcrobaticsPlus”, “Blank”>>
```

可以看到，程序找到了一个打破不变量的反例：

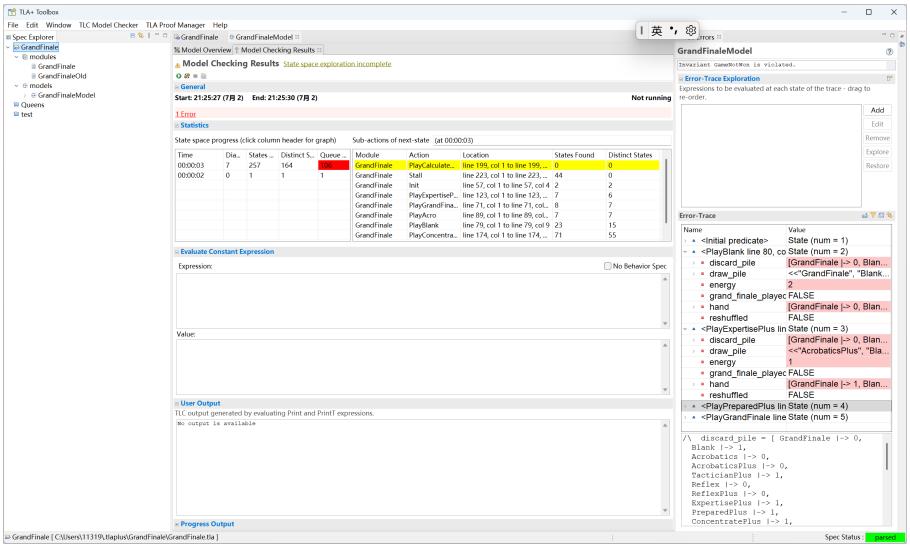


图 3: 模型报错

具体 Error Trace 如下，我们可以从中得到出错步骤为：先打出一张防御，打出一张独门技术 +，打出一张早有准备 +（弃牌为战术大师 + 和全神贯注 +），接下来即可打出华丽收场。

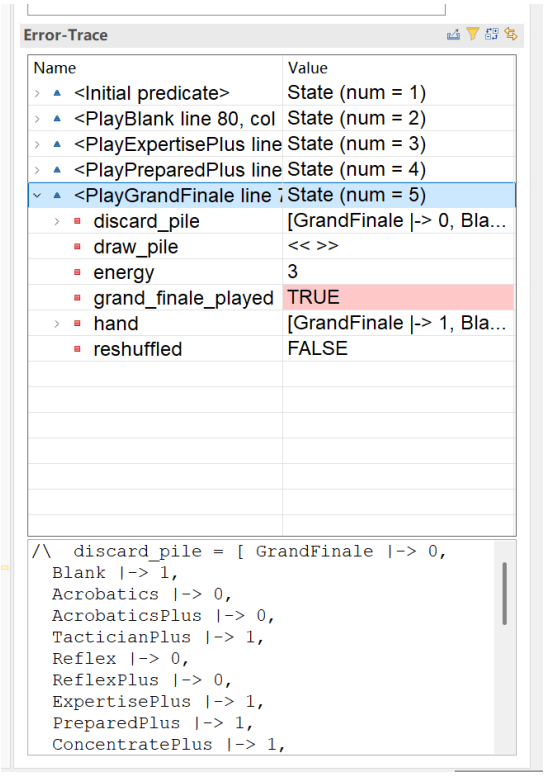


图 4: Error Trace

经测试，成功打出华丽收场！



图 5: 华丽收场

5 考虑洗牌的模型

在前述模型中，为了降低计算量和简化模型，我们选择了暂时不考虑洗牌这一行为，但在实际游戏中，较小的卡组为了打出华丽收场，很有可能在一回合中多次触发洗牌。因此，我们将以一个卡牌数量较少的情况为例，尝试对考虑洗牌的情况进行分析。

主要的代码修改在 DoDraw 函数中，若抽牌堆数量不够，则先拿走当前抽牌堆所有的牌，再查看弃牌堆数量，若弃牌堆足够，通过 PermutationsOfBag 生成弃牌堆的所有可能排列，对每个排列，抽取最靠前的需要数量追加到已抽牌后面，剩余部分成为新的抽牌堆。

```
DoDraw(dp, disc_pile, n, current_hand_size) ==
LET allowed == MaxHandSize - current_hand_size
    T == IF n > allowed THEN allowed ELSE n
IN IF T <= Len(dp) THEN
     { [ drawn |-> SubSeq(dp, 1, T),
         rem_dp |-> SubSeq(dp, T + 1, Len(dp)),
         new_disc |-> disc_pile ] }
   ELSE
     \* 牌库不够，触发洗牌
     LET needed == T - Len(dp)
         avail == BagSize(disc_pile)
         actual_extra == IF needed > avail THEN avail ELSE needed
     IN
     { [ drawn |-> dp \o SubSeq(shuffled_dp, 1, actual_extra),
         rem_dp |-> SubSeq(shuffled_dp, actual_extra + 1, avail),
         new_disc |-> EmptyBag ]   \* 弃牌堆被洗空
       : shuffled_dp \in PermutationsOfBag(disc_pile) }
```

PermutationsOfBag 实现如下，其中 seq 是一个从 1..S 到卡牌名称的函数（即长度为 S 的序列），要求对每种卡牌，它在序列中出现的次数等于 b 中存储的数值。这样即可输出所有可能的序列的集合，这些序列的长度等于总牌数，并且每种卡牌的出现次数恰好等于其在 b 中的数量。

```
PermutationsOfBag(b) ==
    IF BagSize(b) = 0 THEN { <<>> }
    ELSE LET S == BagSize(b)
             Elements == {c \in Cards : b[c] > 0}
         IN { seq \in [1..S -> Elements] :
                \A c \in Elements :
                  Cardinality({i \in 1..S : seq[i] = c}) = b[c] }
```

完整源代码见附件。我们选用一个一定需要触发洗牌才能打出华丽收场的初始状态进行实验：

```
Init ==
    /\ energy = 3
    /\ grand_finale_played = FALSE
    /\ hand = [EmptyBag EXCEPT !["GrandFinale"]=1, !["Acrobatics"]=1, !["
       Reflex"]=1]
```

```
    /\ draw_pile = <<"Blank", "Blank">>
    /\ discard_pile = [EmptyBag EXCEPT ![“Blank”]=3]
```

此时，手牌包括华丽收场，杂技，本能反应，抽牌堆中有两张白板，弃牌堆中有三张白板。

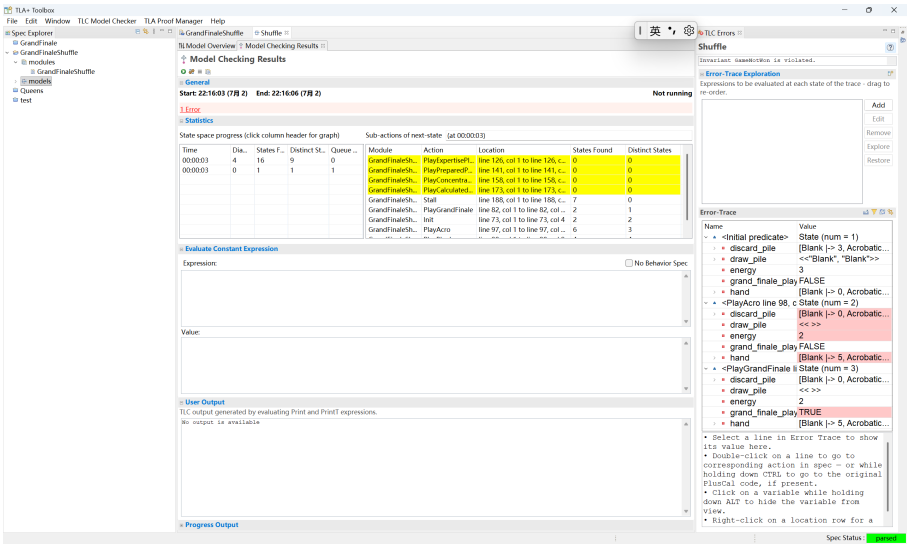


图 6: 考虑洗牌的模型

可以看到，模型成功找到了打出华丽收场的方式。具体流程是先打出一张杂技，此时杂技需要抽三张牌，抽牌堆只有两张牌，数量不足，因此弃牌堆中的三张白板会被洗入抽牌堆。抽牌动作完成后，抽牌堆有两张白板，程序选择弃掉本能反应，再抽两张牌，刚好抽空抽牌堆，打出华丽收场。

对于更加复杂的初始状态如：

```
Init ==
    /\ energy = 3
    /\ grand_finale_played = FALSE
    /\ hand = [EmptyBag EXCEPT ![“GrandFinale”]=1, ![“Acrobatics”]=1, ![“
       TacticianPlus”]=1]
    /\ draw_pile = <<“Blank”, “Blank”>>
    /\ discard_pile = [EmptyBag EXCEPT ![“Blank”]=3, ![“
       CalculatedGamblePlus”]=1, ![“Reflex”]=1]
```

程序仍然能找到打出华丽收场的解。可以看到，Error Trace 包含七步，已经明显超出了我在平时游戏中的思考深度。

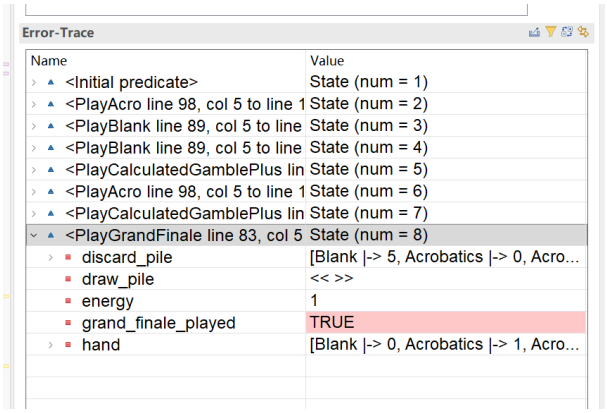


图 7: 复杂情况的 Error Trace

出乎意料的是，在测试模型时，即使面对更复杂的情况，程序用的时间也并没有很夸张，许多情况都能在一分钟内甚至十秒左右给出结果，这是因为 TLC 默认使用广度优先搜索求解，即寻找最短的能够打破不变量的路径；同时，我们在设计代码时的多重集合结构大幅减少了状态冗余，达到了出色的效率。

在 Statistics 面板中，我们可以看到 TLC 探索状态空间的具体数据。其中，Diameter 表示 TLC 从初态出发到找到反例，最多经过了 9 步，States Found 为 TLC 生成的状态节点总数，Distinct States 是去重后的真实状态集大小。右侧的子动作状态拆解展示了每一个具体动作分别贡献了多少状态，可以看到，TLC 找到了 9 个打出华丽收场的状态，其中 8 个是独立状态，这也体现出了形式化方法面对复杂状态机的高效。

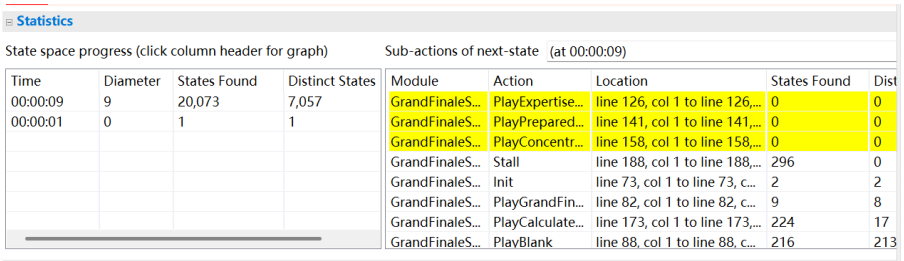


图 8: TLC 的 Statistics 面板

需要注意的是，这样的模型仅仅是证明了在最好情况下解的存在性，即在某个特定的洗牌结果下，存在一种出牌方式可以达成目标，而不是无论洗牌结果如何，都存在一种出牌方式可以达到目标。虽然这样的模型可以作为该局面“有可能获胜”的数学证据，但它确实不能保证在真实游戏的随机洗牌中，玩家一定能成功。如果要证明无论对哪种洗牌序列都能成功，就需要将存在量词改为全称量词，即要求对于 `PermutationsOfBag` 生成的每一个洗牌序列，都能找到一套出牌顺序最终达成目标。这会使状态空间急剧膨胀，而且必须证明一种“策略”能够应对所有随机结果，而不是仅仅存在一条幸运的路径，这会远远超出常规模型检测的复杂度，且很可能需要结合博弈论或策略机等知识，用课上所学的简单形式化方法实现难度较高，因此暂不考虑。

6 实验总结

本实验以 Roguelike 卡牌游戏《杀戮尖塔》中的代表性卡牌“华丽收场”为研究对象，探索了形式化方法在游戏状态可达性分析中的应用。我们将游戏的核心机制——手牌、能量、抽牌堆、弃牌堆等抽象为有限状态机模型，并利用 TLA+ 语言及其模型检测器 TLC 进行验证。

在不考虑洗牌的简化模型中，系统通过多重集合与序列分别建模无序的手牌和有序的抽牌堆，借助存在量词遍历弃牌决策空间，实现了对出牌序列的搜索。实验表明，模型能够针对给定的初始手牌与牌库配置，成功找到触发“华丽收场”的操作序列，证明了形式化方法在复杂卡牌逻辑验证中的可行性。

在此基础上，我们进一步扩展模型以处理包含随机洗牌的情况，通过枚举弃牌堆的所有可能排列来覆盖状态空间。即使面对多步操作和较大分支的洗牌场景，模型依然能够在较短时间内给出正确解法，展现出良好的求解能力。

总体而言，本实验证明了形式化方法能够有效应对卡牌类游戏的场景，这种利用穷举思维与严格逻辑验证系统特性的能力，不仅对平时的游戏颇有帮助，更对游戏平衡性分析等现实工作具有重大意义，也使我收获良多。